You'll probably have a database in mind when you start your application, and you'll be pleased to know that EF Core has most of the popular ones covered. Adding support for a given database involves adding the correct NuGet package to your .csproj file, such as the following:

- *PostgreSQL*—Npgsql.EntityFrameworkCore.PostgreSQL
- *Microsoft SQL Server*—Microsoft.EntityFrameworkCore.SqlServer
- *MySQL*—MySql.Data.EntityFrameworkCore
- *SQLite*—Microsoft.EntityFrameworkCore.SQLite

Some of the database provider packages are maintained by Microsoft, some are maintained by the open-source community, and some (such as the Oracle provider) require a paid license, so be sure to check your requirements. You can find a list of providers at https://docs.microsoft.com/ef/core/providers.

You install a database provider in your application in the same way as any other library: by adding a NuGet package to your project's .csproj file and running `dotnet restore` from the command line (or letting Visual Studio automatically restore for you).

EF Core is inherently modular, so you'll need to install multiple packages. I'm using the SQLite database provider, so I'll be using the SQLite packages:

- *Microsoft.EntityFrameworkCore.SQLite*—This package is the main database provider package for using EF Core at runtime. It also contains a reference to the main EF Core NuGet package.
- *Microsoft.EntityFrameworkCore.Design*—This package contains shared build-time components for EF Core, required for building the EF Core data model for your app.